

AEM Architecture Interview Cheat Sheet (OSGi, Sling, JCR)

Prepared for: Interview-focused learning and quick revision

1. OSGi Basics

What is OSGi and why is it used in AEM?

OSGi (Open Services Gateway Initiative) is a modular framework for Java applications. AEM uses OSGi to dynamically manage bundles (start/stop/update without full restart), provide dependency injection via the Service Registry, and keep features decoupled and configurable.

Create an OSGi Service (Declarative Services)

```
@Component(service = MyService.class)
public class MyServiceImpl implements MyService {
    @Override
    public String getMessage() {
        return "Hello from OSGi Service!";
    }
}
```

Inject the Service in Another Class

```
@Reference
private MyService myService;
```

@Reference vs @Inject

@Reference injects OSGi services via the Service Registry; @Inject is used in Sling Models to inject adaptable values (resources, requests, ValueMap properties).

2. Sling Framework

How Sling resolves /content/site/page.print.html

Sling maps the URL to a JCR resource (/content/site/page), reads the resource type to select the rendering script (e.g., page.html). The selector 'print' modifies the rendering logic (e.g., print-specific script or behavior).

Create a Sling Model

```
@Model(adaptables = Resource.class)
public class MyModel {
    @Inject
    private String title;

    public String getTitle() {
        return title;
    }
}
```

```
}  
}
```

Sling Models vs WCMUsePojo

Sling Models are annotation-driven, support dependency injection, and are cleaner. WCMUsePojo requires extending a base class and overriding activate(); Sling Models are preferred in modern AEM projects.

Sling Selectors, Extensions, and Script Resolution

Selectors appear between the resource name and extension (e.g., page.print.html). Extensions (html, json) choose output format. Script resolution follows resource type, matching scripts under /apps/ with priority rules.

3. JCR Basics

How content is stored in AEM

AEM uses a JCR-compliant repository (Apache Jackrabbit Oak). Content is hierarchical: nodes and properties. Common node types include cq:Page (page), cq:Component (component), and nt:unstructured (flexible nodes).

JCR SQL2 Query Example

```
String sql2 = "SELECT * FROM [cq:Page] WHERE ISDESCENDANTNODE('/content/site')";  
QueryManager qm = session.getWorkspace().getQueryManager();  
Query q = qm.createQuery(sql2, Query.JCR_SQL2);  
NodeIterator nodes = q.execute().getNodes();  
while (nodes.hasNext()) {  
    Node n = nodes.nextNode();  
    // process node  
}
```

nt:unstructured vs cq:Page

nt:unstructured is a generic, schema-flexible node type; cq:Page is a structured node type used by AEM to represent pages with jcr:content child holding properties and components.

Versioning and Observation

JCR supports versioning to track changes and observation to listen to repository events. Use these features for audit trails and content-driven triggers.

4. AEM Request Lifecycle: OSGi + Sling + JCR

JCR stores the content; Sling resolves HTTP requests to resources and chooses scripts based on resource type; OSGi provides backend services and configuration. The dispatcher caches and forwards requests to publish instances for performance.

Common Interview Scenarios

- Write an OSGi service and consume it in a Sling Model.
- Create a servlet bound to a resource type and return JSON.
- Optimize a SQL2 query and explain indexing in Oak.
- Debug a failing bundle via the OSGi

Web Console (components/services/configs).

5. Sample Codes

Resource-Type Servlet (GET JSON)

```
@Component(service = Servlet.class,
            property = {
                "sling.servlet.resourceTypes=example/components/page",
                "sling.servlet.methods=GET",
                "sling.servlet.extensions=json"
            })
public class ExampleServlet extends SlingSafeMethodsServlet {
    @Reference
    private MyService myService;

    @Override
    protected void doGet(SlingHttpServletRequest request, SlingHttpServletResponse response)
        throws ServletException, IOException {
        response.setContentType("application/json");
        response.getWriter().write("{\"message\": \"" + myService.getMessage() + "\"}");
    }
}
```

Sling Model consuming OSGi Service

```
@Model(adaptables = Resource.class)
public class GreetingModel {
    @Reference
    private MyService myService;

    public String getGreeting() {
        return myService.getMessage();
    }
}
```

Basic JCR CRUD (ResourceResolver -> Session)

```
ResourceResolver resolver = request.getResourceResolver();
Session session = resolver.adaptTo(Session.class);
Node root = session.getRootNode();
Node node = root.addNode("example", "nt:unstructured");
node.setProperty("title", "Hello");
session.save();
```

6. Dispatcher Quick Notes

Dispatcher caches rendered pages and assets and forwards uncached requests to publish instances. Key topics: cache invalidation (flush agents), cache rules, filters, and security (whitelisting, denying sensitive paths).

7. Rapid-Fire Q&A;

Selectors vs Extensions

Selectors sit before the extension and influence rendering variants; extensions primarily denote output type (html, json).

Sling Resource Types

Define how a resource is rendered; scripts live under /apps/.

HTL vs JSP

HTL (Sightly) is secure and recommended; JSP is legacy.

OSGi Configs

Use OSGi Console or code-based `@ObjectClassDefinition` with `.cfg/.config` stored under /apps or /conf.

Oak Indexes

Create Lucene indexes for performant SQL2/QueryBuilder queries.

8. Mini Reference: Consoles & Paths

• OSGi Web Console: /system/console • Sling Resource Resolver: /system/console/sling • Components: /apps, /libs • Content: /content • Configs: /conf, /apps//osgiconfig

Interview Checklist

✓ Understand request flow (URL → Resource → Script) ✓ Be able to write a DS service and a Sling Model ✓ Know SQL2 basics and when to index ✓ Explain dispatcher caching and invalidation